High-Performance Distributed Computing and Inference Optimizations

Sapienza University of Rome
14-15 April 2026

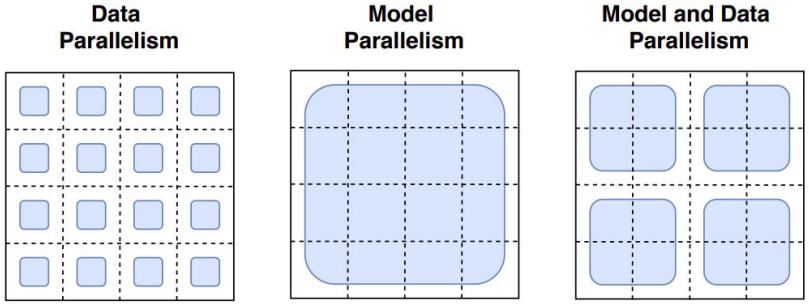
Sergio Orlandini
CINECA
s.orlandini@cineca.it

Parallelization Schemas

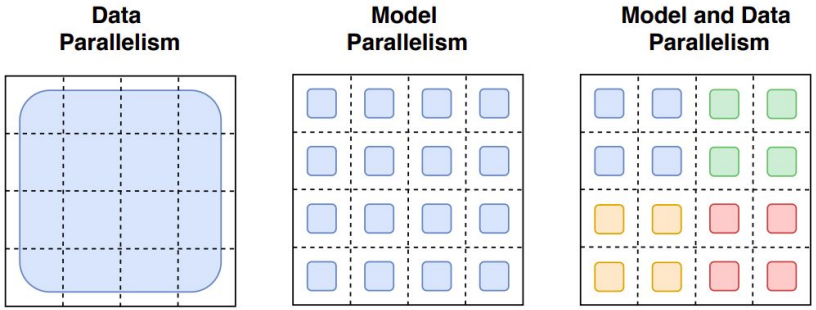
- **Data** Parallelism
  - The **module is replicated** on each processing element (PE) and each replica handles a **chunk of the data**.
  - No communication is needed until the forward and backward passes are finished.
  - Then the gradients need to be then aggregated across each replica.
- **Model** Parallelism
  - It is necessary when the model doesn't fit on a single processing element.
  - Model parallelism **shards a model** across multiple processing elements.
  - Communications are needed during the forward and backward passes.
  - **Tensor** Parallelism
    - Shards parameters of single layers with huge number of parameters across processing elements.
  - **Pipeline** Parallelism
    - Different layers are splitted across processing elements and pipelining is used to run all the PEs simultaneously.
    - The pipelines can be processed in parallel improving both memory and computation of training.
- **3D** Parallelism
  - Data + Tensor + Pipeline Parallelisms

Parallelization Schemas

How the **model** is split



How the **data** is split



source: “Switch Transformers” Fedus et al., 2021

Distributed Parallel Training/Inference

Several frameworks and packages can be used to parallelize DL frameworks:

- PyTorch
- Tensorflow
- Horovod
- Accelerate
- Deepspeed
- PyTorch Lightning

...

PyTorch

Different strategy can be used in PyTorch to implement Data Parallelism Paradigm.

Best Practice:

- Use single GPU if the data and model fit in one GPU.
  PyTorch only use one GPU by default.
- Use DataParallel module to speed up training on a multi-GPU node.
- Use DistributedDataParallel module to speed up training on a multi node/GPU system.
- Use multi-GPU FullyShardedDataParallel training on a multi node/GPU system when the data and model cannot fit on a single GPU.

DataParallel

`DataParallel` module **automatically splits** the data and sends it to multiple replicas of the model on multiple GPUs.
After each GPU has finished, DataParallel module collects and merges the results.

```
model = Model(input_size, output_size)

if torch.cuda.device_count() > 1:
  model = torch.nn.DataParallel(model)

model.to(device)

# select only some gpus
model = torch.nn.DataParallel(model, device_ids=[0, 1, 2])
```

```
$ python training.py
```

DataParallel: Simple Example

- Model
  - input size = 8
  - output size = 4
- Data
  - data size = 100
  - batch size = 32
  - num. batches = 4

```
Model(
  (fc): Linear(in_features=8, out_features=4, bias=True)
)


Single GPU:

DATA: input torch.Size([32, 8]) -> output torch.Size([32, 4])
   MODEL GPU 0 : input torch.Size([32, 8]) -> output torch.Size([32, 4])
DATA: input torch.Size([32, 8]) -> output torch.Size([32, 4])
   MODEL GPU 0 : input torch.Size([32, 8]) -> output torch.Size([32, 4])
DATA: input torch.Size([32, 8]) -> output torch.Size([32, 4])
   MODEL GPU 0 : input torch.Size([32, 8]) -> output torch.Size([32, 4])
DATA: input torch.Size([4, 8]) -> output torch.Size([4, 4])
   MODEL GPU 0 : input torch.Size([4, 8]) -> output torch.Size([4, 4])
```

```
DataParallel(
  (module): Model(
    (fc): Linear(in_features=8, out_features=4, bias=True)
  )
)


DataParallel with 4 GPUs:

DATA: input torch.Size([32, 8]) -> output torch.Size([32, 4])
   MODEL GPU 0 : input torch.Size([8, 8]) -> output torch.Size([8, 4])
   MODEL GPU 1 : input torch.Size([8, 8]) -> output torch.Size([8, 4])
   MODEL GPU 2 : input torch.Size([8, 8]) -> output torch.Size([8, 4])
   MODEL GPU 3 : input torch.Size([8, 8]) -> output torch.Size([8, 4])
DATA: input torch.Size([32, 8]) -> output torch.Size([32, 4])
   MODEL GPU 0 : input torch.Size([8, 8]) -> output torch.Size([8, 4])
   MODEL GPU 1 : input torch.Size([8, 8]) -> output torch.Size([8, 4])
   MODEL GPU 2 : input torch.Size([8, 8]) -> output torch.Size([8, 4])
   MODEL GPU 3 : input torch.Size([8, 8]) -> output torch.Size([8, 4])
DATA: input torch.Size([32, 8]) -> output torch.Size([32, 4])
   MODEL GPU 0 : input torch.Size([8, 8]) -> output torch.Size([8, 4])
   MODEL GPU 1 : input torch.Size([8, 8]) -> output torch.Size([8, 4])
   MODEL GPU 2 : input torch.Size([8, 8]) -> output torch.Size([8, 4])
   MODEL GPU 3 : input torch.Size([8, 8]) -> output torch.Size([8, 4])
DATA: input torch.Size([4, 8]) -> output torch.Size([4, 4])
   MODEL GPU 0 : input torch.Size([1, 8]) -> output torch.Size([1, 4])
   MODEL GPU 1 : input torch.Size([1, 8]) -> output torch.Size([1, 4])
   MODEL GPU 2 : input torch.Size([1, 8]) -> output torch.Size([1, 4])
   MODEL GPU 3 : input torch.Size([1, 8]) -> output torch.Size([1, 4])
```

DistributedDataParallel (aka DDP)

- `DistributedDataParallel` does not split the input between GPUs.
- The user is responsible for distributing the data. A common approach is to use a `DistributedSampler`.
- A initialization procedure is needed by calling `torch.distributed.init_process_group()`.
- nccl backend is currently the fastest and highly recommended backend when using GPUs. This applies to both single-node and multi-node distributed training.

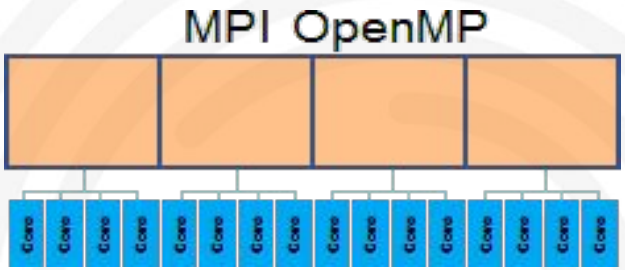
DistributedDataParallel: SLURM submission

```
4 processes per node, each process control 1 GPU and 8 cpus per process.
NB: Switch on the binding of the OMP threads and map the MPI processes with the --map-by option


     #SBATCH --ntasks-per-node=4        # 4 processes per node
     #SBATCH --cpus-per-task=8          # 8 cores per process
     #SBATCH --gres=gpu:4               # 4 gpus per node

     export OMP_NUM_THREADS=8

     mpirun --map-by socket:PE=8 python <your_script>.py
```




```
$ mpirun -n 4 --map-by socket:PE=8 --report-bindings python <your_script>.py

rank 0 [core  0,  1,  2,  3,  4,  5,  6,  7] [B/B/B/B/B/B/B/B/./././././././././././././././././././././././././.]
rank 1 [core  8,  9, 10, 11, 12, 13, 14, 15] [./././././././././B/B/B/B/B/B/B/B/././././././././././././././././.]
rank 2 [core 16, 17, 18, 19, 20, 21, 22, 23] [././././././././././././././././././B/B/B/B/B/B/B/B/./././././././.]
rank 3 [core 24, 25, 26, 27, 28, 29, 30, 31] [./././././././././././././././././././././././././././././B/B/B/B/B/B/B/B]
...
```

DistributedDataParallel: SLURM submission

```
#SBATCH --nodes 2              # 2 nodes
#SBATCH --ntasks-per-node=4    # 4 processes per node
#SBATCH --cpus-per-task=8      # 8 cores per process
#SBATCH --gres=gpu:4           # 4 gpus per node

mpirun --map-by socket:PE=8 <your_exe>
```

```
$ sbatch SCRIPT.slurm

i am rank 0 of 8, local_rank 0
i am rank 1 of 8, local_rank 1
i am rank 2 of 8, local_rank 2
i am rank 3 of 8, local_rank 3
i am rank 4 of 8, local_rank 0
i am rank 5 of 8, local_rank 1
i am rank 6 of 8, local_rank 2
i am rank 7 of 8, local_rank 3
```

world size	rank	local size	local rank	gpu_id
8	0	4	0	0
8	1	4	1	1
8	2	4	2	2
8	3	4	3	3
8	4	4	0	0
8	5	4	1	1
8	6	4	2	2
8	7	4	3	3

DistributedDataParallel: Initialization

```
torch.distributed.init_process_group(
    backend='nccl',                 # backend used to communicate
    rank=rank,                      # rank (id) of the current process
    world_size=num_process          # total number of processes
)

model = torch.nn.parallel.DistributedDataParallel(
    model,
    device_ids=[local_rank],  # list of devices per process (input)
    output_device=local_rank  # device location for output
)

torch.cuda.set_device(local_rank)  # each process control one GPU
```

torch.distributed for any Launcher

```python
def get_resources():

    if os.environ.get("RANK"):
        # launched with torchrun (python -m torch.distributed.run)
        rank = int(os.getenv("RANK"))
        local_rank = int(os.getenv("LOCAL_RANK"))
        world_size = int(os.getenv("WORLD_SIZE"))
        local_size = int(os.getenv("LOCAL_WORLD_SIZE"))

    elif os.environ.get("OMPI_COMMAND"):
        # launched with mpirun
        rank = int(os.environ["OMPI_COMM_WORLD_RANK"])
        local_rank = int(os.environ["OMPI_COMM_WORLD_LOCAL_RANK"])
        world_size = int(os.environ["OMPI_COMM_WORLD_SIZE"])
        local_size = int(os.environ["OMPI_COMM_WORLD_LOCAL_SIZE"])

    else:
        # launched with srun (SLURM)
        rank = int(os.environ["SLURM_PROCID"])
        local_rank = int(os.environ["SLURM_LOCALID"])
        world_size = int(os.environ["SLURM_NPROCS"])
        local_size = int(os.environ["SLURM_NTASKS_PER_NODE"])

    num_workers = int(os.environ["SLURM_CPUS_PER_TASK"])

    return rank, local_rank, world_size, local_size, num_workers

rank, local_rank, world_size, local_size, num_workers = get_resources()
```

DistributedDataParallel: Data Distribution

```
from torch.utils.data import DataLoader
from torch.utils.data.distributed import DistributedSampler

sampler = DistributedSampler(dataset=dataset)

loader = DataLoader(
    dataset,
    batch_size=batch_size,
    shuffle=False,
    num_workers=num_workers,
    pin_memory=True,
    sampler=sampler)
```

DistributedDataParallel: Complete Example

```
...

rank, local_rank, world_size, local_size, num_workers = get_resources()

torch.distributed.init_process_group("nccl", rank=rank, world_size=world_size)

torch.cuda.set_device(local_rank)

...

sampler = torch.utils.data.distributed.DistributedSampler(dataset=dataset)

loader = torch.utils.data.DataLoader(
    dataset, batch_size=batch_size, shuffle=False, num_workers=num_workers, pin_memory=True, sampler=sampler
)

model = torch.nn.parallel.DistributedDataParallel(model, device_ids=[local_rank], output_device=local_rank)

...

for epoch in range(epochs):
   for images, target in loader:
       # move data to the same device as model
       images = images.to(device, non_blocking=True)
       target = target.to(device, non_blocking=True)

       train(model, images, target)
```

DistributedDataParallel: Complete SLURM Script

```
#!/bin/bash
#SBATCH --nodes=2                # number of nodes
#SBATCH --ntasks-per-node=4      # number of process per node
#SBATCH --cpus-per-task=8        # number of threads per process
#SBATCH --gres=gpu:4             # number of gpus per node
#SBATCH --time 08:00:00          # format: HH:MM:SS

# the node with rank 0 is used to set up all connections using a free port
export MASTER_ADDR=$(scontrol show hostnames "$SLURM_JOB_NODELIST" | head -n 1)
export MASTER_PORT=12340

export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK

mpirun -n 8 --map-by socket:PE=8 --report-bindings python SCRIPT.py

# or

srun python SCRIPT.py
```

Fully Shared Data Parallel

- `DistributedDataParallelism` requires that the model fits in a single GPU.
- Recent approaches like *Fully Sharded Data Parallel* from FairScale and *ZeRO* from DeepSpeed overcome this limit sharding model parameters, gradients and optimizer states between GPUs.
- FullySharedDataParallel (FSDP) implements a type of data parallelism which shards model's parameters, gradient and optimizer states between GPUs and moreover can also offload sharded model parameters to CPUs.
- In FSDP the model can be wrapped in automatic or manual mode. The *auto* wrapping mode is a drop-in replacement for DDP without changing the source code. While in *manual* wrapping more complex wrapping strategy can be used.

Fully Shared Data Parallel

```
...

rank, local_rank, world_size, local_size, num_workers = get_resources()

torch.distributed.init_process_group("nccl", rank=rank, world_size=world_size)

torch.cuda.set_device(local_rank)

...

sampler = torch.utils.data.distributed.DistributedSampler(dataset=dataset)

loader = torch.utils.data.DataLoader(
    dataset, batch_size=batch_size, shuffle=False, num_workers=num_workers, pin_memory=True, sampler=sampler
)

model = torch.nn.parallel.DistributedDataParallel(model, device_ids=[local_rank], output_device=local_rank)

model = torch.distributed.fsdp.FullyShardedDataParallel(
    model(),
    fsdp_auto_wrap_policy=default_auto_wrap_policy,
    cpu_offload=CPUOffload(offload_params=True)
)

...
```

TensorFlow Distributed Training

`tf.distribute.Strategy` is a TensorFlow module to distribute training across multiple GPUs.

- Types of Strategies:
  - `MirroredStrategy`
  - `MultiWorkerMirroredStrategy`
  - `CentralStorageStrategy`
  - `ParameterServerStrategy`
- Synchronous vs Asynchronous Training:
  - **Synchronous**: GPUs train over different chunk of input data in sync and aggregate gradients at each step via all-reduce operations.
  - **Asynchronous**: GPUs are independently training over input data and updating variables asynchronously.
- Hardware platform
  - Training on multi GPUs systems or multi nodes/multi GPUs systems or on Cloud TPUs.

MirroredStrategy

`MirroredStrategy` is used for synchronous training across multiple replicas.

It will replicate the variables across all the replicas and keep them in sync using an all-reduce algorithm.

MirroredStrategy takes care of replicating the model, aggregating gradients, and more.

It is used to distributed training on multiple GPUs on one machine.

NB1: The batch size should be increased with GPU number. (use `strategy.num_replicas_in_sync` to get the number of replica)

NB2: Tune the learning rate with the number of GPUs .

```
strategy = tf.distribute.MirroredStrategy(["GPU:0", "GPU:1"])
```

MirroredStrategy

How does `MirroredStrategy` work?

- All the variables and the model are replicated across the replicas.
- Input is distributed across the replicas.
- Each replica calculates the loss and the gradients.
- The gradients are synchronized across all the replicas by summing them.
- After the sync, the same update is made to the copies of the variables on each replica.

MirroredStrategy / Keras

```
strategy = tf.distribute.MirroredStrategy(["GPU:0", "GPU:1"])

with strategy.scope():
  model = tf.keras.Sequential([
      tf.keras.layers.Conv2D(32, 3, activation='relu', input_shape=(28, 28, 1)),
      tf.keras.layers.MaxPooling2D(),
      tf.keras.layers.Flatten(),
      tf.keras.layers.Dense(64, activation='relu'),
      tf.keras.layers.Dense(10)
  ])

  model.compile(loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
                optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
                metrics=['accuracy'])

dataset = dataset.batch(batch_size * mirrored_strategy.num_replicas_in_sync)

model.fit(train_dataset, epochs=10)
```

MirroredStrategy / TensorFlow

```
strategy = tf.distribute.MirroredStrategy()

with strategy.scope():
    model = ...
    optimizer = ...

train_dist_dataset = strategy.experimental_distribute_dataset(train_dataset)

global_batch_size = batch_size * strategy.num_replicas_in_sync

...

@tf.function
def distributed_train_step(dataset_inputs):
  per_replica_losses = strategy.run(train_step, args=(dataset_inputs,))
  return strategy.reduce(tf.distribute.ReduceOp.SUM, per_replica_losses, axis=None)

for epoch in range(num_epochs):
  total_loss = 0.0
  num_batches = 0
  for x in train_dist_dataset:
    total_loss += distributed_train_step(x)
    num_batches += 1
  train_loss = total_loss / num_batches
```

MultiWorkerMirroredStrategy

`MultiWorkerMirroredStrategy` implements synchronous distributed training across multiple workers, each with multiple GPUs.

Similar to `MirroredStrategy`, it creates copies of all variables in the model on each device across all workers.

All workers train over different chunk of input data in sync, and at each step the gradients are aggregated.

`MultiWorkerMirroredStrategy` takes care of replicating the model and aggregating gradients.

The dataset sharding policy can be selected. If not selected the default autosharding is used.

```
strategy = tf.distribute.MultiWorkerMirroredStrategy()
```

MultiWorkerMirroredStrategy Example

```
tf_config = {
    'cluster': {
        'worker': ['lrdn1342:12345', 'lrdn1343:23456']
    },
    'task': {'type': 'worker', 'index': 0} # 'chief' worker
}
os.environ['TF_CONFIG'] = json.dumps(tf_config) # serialize tf_config to json

strategy = tf.distribute.MultiWorkerMirroredStrategy()

with strategy.scope():
  model = tf.keras.Sequential([
      ...
  ])

  model.compile(loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
                optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
                metrics=['accuracy'])

model.fit(train_dataset, epochs=10)
```

Horovod Distributed Learning

Horovod is a distributed deep learning training framework for TensorFlow, Keras, PyTorch, and Apache MXNet.

The goal of Horovod is to make distributed deep learning fast and easy to use.

1. Initialize Horovod with `hvd.init()`
2. Bind each GPU to a single process
3. Scale the learning rate by the number of workers
4. Wrap the optimizer in `hvd.DistributedOptimizer`
5. Broadcast the initial variable states from rank 0 to all other processes
6. Modify your code to save checkpoints only on worker 0 to prevent other workers from corrupting them

Horovod Example

```
import horovod.torch as hvd

hvd.init()  # Initialize Horovod

torch.cuda.set_device(hvd.local_rank())  # bind GPU to local rank

...

# Partition dataset among workers using DistributedSampler
train_sampler = torch.utils.data.distributed.DistributedSampler(
    train_dataset, num_replicas=hvd.size(), rank=hvd.rank()
)

train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size, sampler=train_sampler)

...

optimizer = optim.SGD(model.parameters())

# Horovod Distributed Optimizer
optimizer = hvd.DistributedOptimizer(optimizer, named_parameters=model.named_parameters())

# Broadcast parameters from rank 0 to all processes
hvd.broadcast_parameters(model.state_dict(), root_rank=0)

...
```

Accelerate

Accelerate is a library develop by Hugging Face that enables PyTorch code to be run across any distributed configuration in a simple way.

How does Accelerate work?

1. Import and create an `Accelerator` object.
   The `Accelerator` automatically detect the distributed setup and initialize all the components.
   NB: The copy of the model to device is not needed.
2. Pass all the training objects to the `prepare` method.
   The model, the optimizer and the data loaders.
3. Replace the `loss.backward()` with the Accelerate's `backward` method in your training loop.

```
from accelerate import Accelerator
accelerator = Accelerator()
```

Accelerate Training Example

```python
from accelerate import Accelerator
accelerator = Accelerator()

model, optimizer, dataloader = accelerator.prepare(
    model, optimizer, dataloader
)

for epoch in range(num_epochs):
  for inputs, targets in dataloader:
      optimizer.zero_grad()
      outputs = model(inputs)
      loss = loss_function(outputs, targets)
      accelerator.backward(loss)  # replace: loss.backward()
      optimizer.step()
```

Accelerate Launching Script

`mpirun` executable can be used to run the training script:

```
$ mpirun -np 4 python train.py
```

Accelerate also provides an optional CLI tool to configure your training environment.

```
$ accelerate config
$ accelerate launch train.py
```

DeepSpeed

- Developed by Microsoft's AI at Scale initiative.
- The DeepSpeed Library is a wrapper to PyTorch.
- DeepSpeed provides memory-efficient data parallelism and enables training models without model parallelism.
- DeepSpeed supports data parallelism, model parallelism, pipeline parallelism and their combinations.
- Train/Inference models with billions or trillions of parameters.
- DeepSpeed manages multiple techniques, such as: distributed training, mixed precision, gradient accumulation, and checkpoints.
- Excellent scale to thousands GPUs.

DeepSpeed Example

```
model, optimizer, dataloader, _ = deepspeed.initialize(
    args,
    model=model,
    model_parameters=params,
    training_data=train_dataset,
)

deepspeed.init_distributed()

for epoch in range(num_epochs):
  for input, labels in dataloader:

    inputs = inputs.to(model.device)
    labels = labels.to(model.device)

    outputs = model(inputs)
    loss = criterion(outputs, labels)

    model.backward(loss)
    model.step()
```

DeepSpeed Configuration

DeepSpeed features can be configured using a config JSON file.

The configuration file should be specified as `args.deepspeed_config`

see here for a full list of options.

```
{
  "train_batch_size": 8,
  "gradient_accumulation_steps": 1,
  "optimizer": {
    "type": "Adam",
    "params": {
      "lr": 0.00015
    }
  },
  "fp16": {
    "enabled": true
  },
  "zero_optimization": true
}
```

DeepSpeed Launch Script

DeepSpeed supports multi-GPUs and multi-nodes training/inference.

The computing resources should be defined with a hostfiles compatible with OpenMPI.

```
lrdn1342 slots=4 # Num. GPUs available
lrdn1343 slots=4
```

```
deepspeed --hostfile=hostfile \
          training_script.py \
          <args> \
          --deepspeed \
          --deepspeed_config ds_config.json
```